

Chapter-4:

Operating System Basics

Syeda Jannatul Naim
Lecturer | Department of CSE

February, 2026

Content

- 1. What is an operating system**
- 2. Abstract view of Operating System**
- 3. Components of Operating System**
- 4. Program and Process**
- 5. Key functions(services) of an operating system**
- 6. Importance of operating system**
- 7. Types of operating systems**
- 8. Evolution of operating systems**
- 9. Why Operating System Knowledge Is Essential**

1. What Is an Operating System?

❖ **The Core Software**

The master program that manages all hardware and software resources. It starts when we power on (boot) our computing device and runs continuously.

❖ **The Bridge**

Acts as an intermediary between us (the user) or our applications and the complex computer hardware. We don't speak binary; the OS does.

❖ **Provider of Services**

Handles essential background tasks like file saving, network connectivity, printing documents, and enforcing security.

1.2. Popular Operating System



DESKTOP

Microsoft Windows

The standard for pharmacy workstations. High compatibility with legacy dispensing hardware.

Pharmacy Use:

Main dispensing computers, insurance billing, patient record management (EHR).



DESKTOP

macOS

Known for stability and security. Less common in dispensing but popular in admin.

Pharmacy Use:

Hospital administration, medical research, educational tools.



SERVER

Linux

Open-source, secure, and stable. Runs the "back-end" infrastructure invisibly.

Pharmacy Use:

Database servers for drug interactions, MRI/CT scanners, embedded devices.



MOBILE

Android

Versatile mobile OS running on various hardware from phones to scanners.

Pharmacy Use:

Handheld inventory scanners, delivery driver tablets, point-of-sale systems.



MOBILE

iOS / iPadOS

High security and consistent interface. Very popular for clinical mobility.

Pharmacy Use:

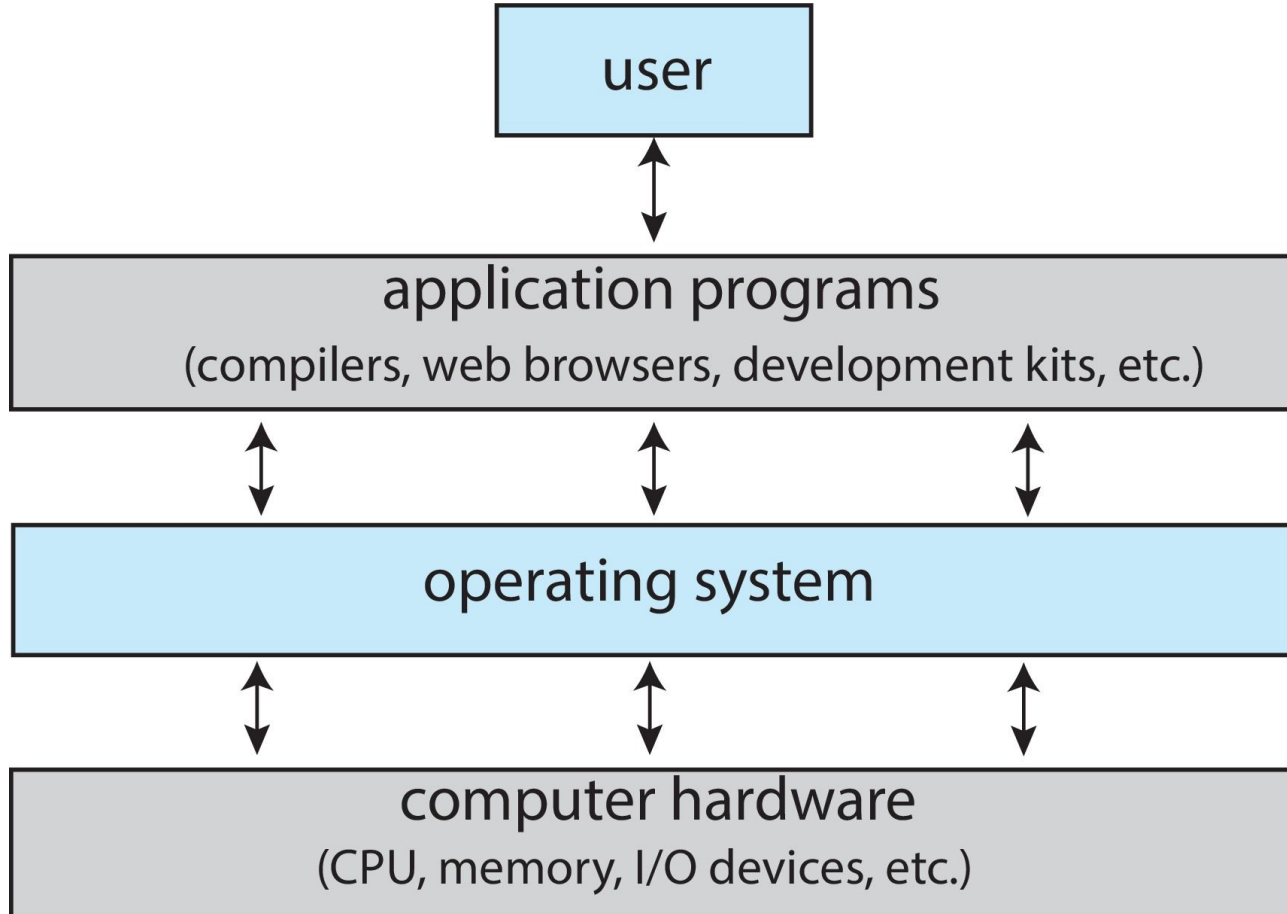
Bedside charting (eMAR), secure messaging apps, drug reference apps.

Interconnection

Modern pharmacy relies on all of these together.

Example: A pharmacy could use workstations on Windows, servers on Linux, and mobile devices on Android or iOS. All of these systems would be interconnected via a Windows PC.

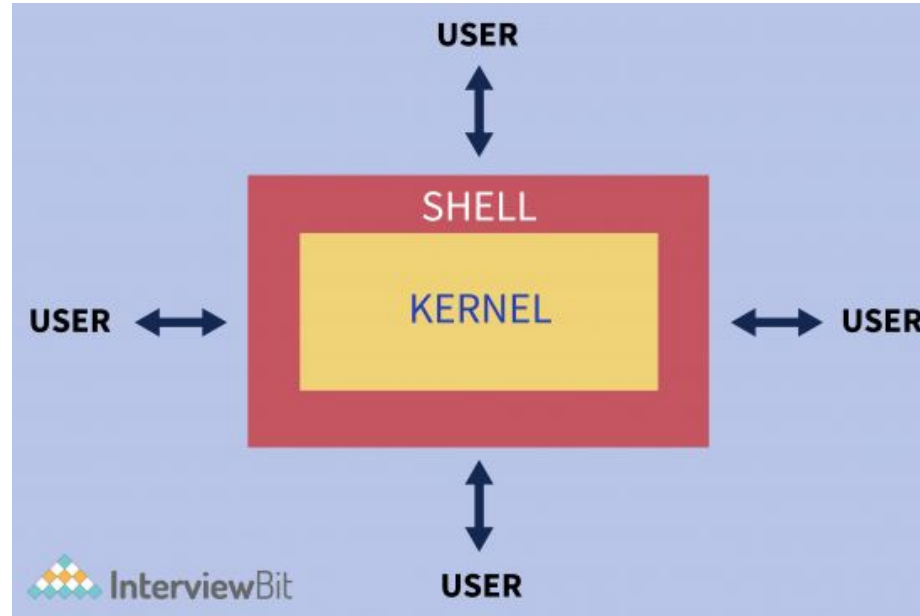
2. Abstract view of Operating System:



3. Components of Operating System

The operating system consists of two components:

1. Shell
2. Kernel



3. Components of Operating System

Kernel: A kernel is an operating system's heart. Its job is to keep track of the hardware and computer's operations. By accessing the different resources of a computer, such as the I/O devices, CPU, and several other resources, a kernel simply works as a bridge between any user and the various resources supplied by a system.

Shell: It refers to a specific user software or environment that gives a user an interface to access the operating system's services. A shell runs several programmes based on the input it receives from the user. Shell commands such as mkdir, ls, and others can be used to ask the operating system to complete a specified action.

3. Components of Operating System

Aspect	Kernel	Shell
Definition	Core part of the operating system	User interface to access OS services
Position in OS	Closest to the hardware	Lies between user and kernel
Main Function	Manages CPU, memory, devices, and processes	Interprets user commands and sends them to the kernel
User Interaction	No direct interaction with users	Direct interaction with users
Access Level	Runs in privileged (kernel) mode	Runs in user mode
Hardware Access	Direct access to hardware	No direct hardware access
Criticality	Essential for OS functioning	OS can run without a shell (but hard to use)
Examples	Linux kernel, Windows NT kernel	Bash, PowerShell, Zsh

4. Program and Process

- A **program** is a passive, static set of instructions stored on a disk (e.g., an `.exe` file), while a **process** is an active, dynamic instance of that program currently executing in memory and using CPU resources.
- **Programs** have long lifespans, whereas **processes** are temporary, created when a program runs and terminated upon completion.
- **Example:** A `calculator.exe` file sitting on your desktop is a program. When you double-click it to start calculating, the Operating System creates a process for it in memory.

5.Key functions(services) of an operating system



Process Management

Creates, schedules, and terminates running programs (processes).



Memory Management

Allocates RAM to apps and prevents them from interfering with each other.



File System

Organizes data into files/folders and manages storage space.



Device (I/O) Mgmt

Communicates with hardware like printers, scanners, and keyboards.



CPU Scheduling

Decides which task gets to use the processor and for how long.



Security

Manages user logins, permissions, and protects against unauthorized access.



Networking

Handles data transmission over Wi-Fi, Ethernet, and the Internet.



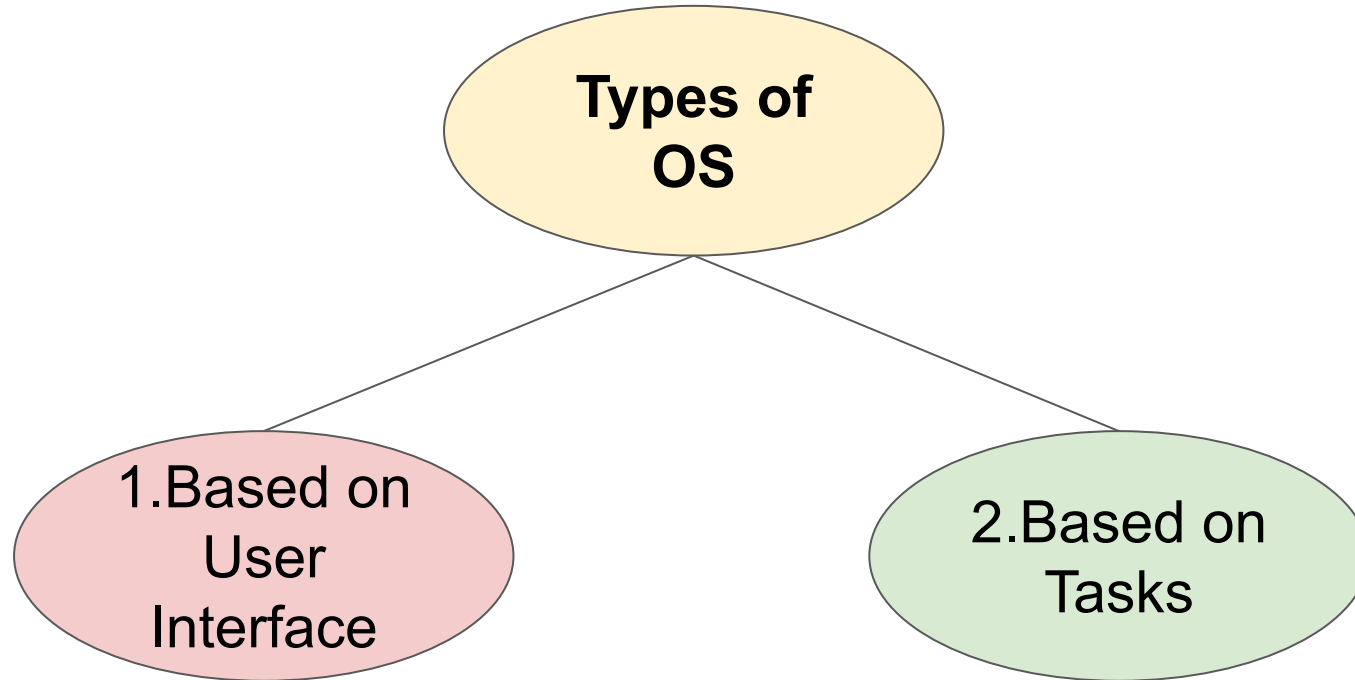
Error Handling

Detects system crashes or hardware faults and logs them.

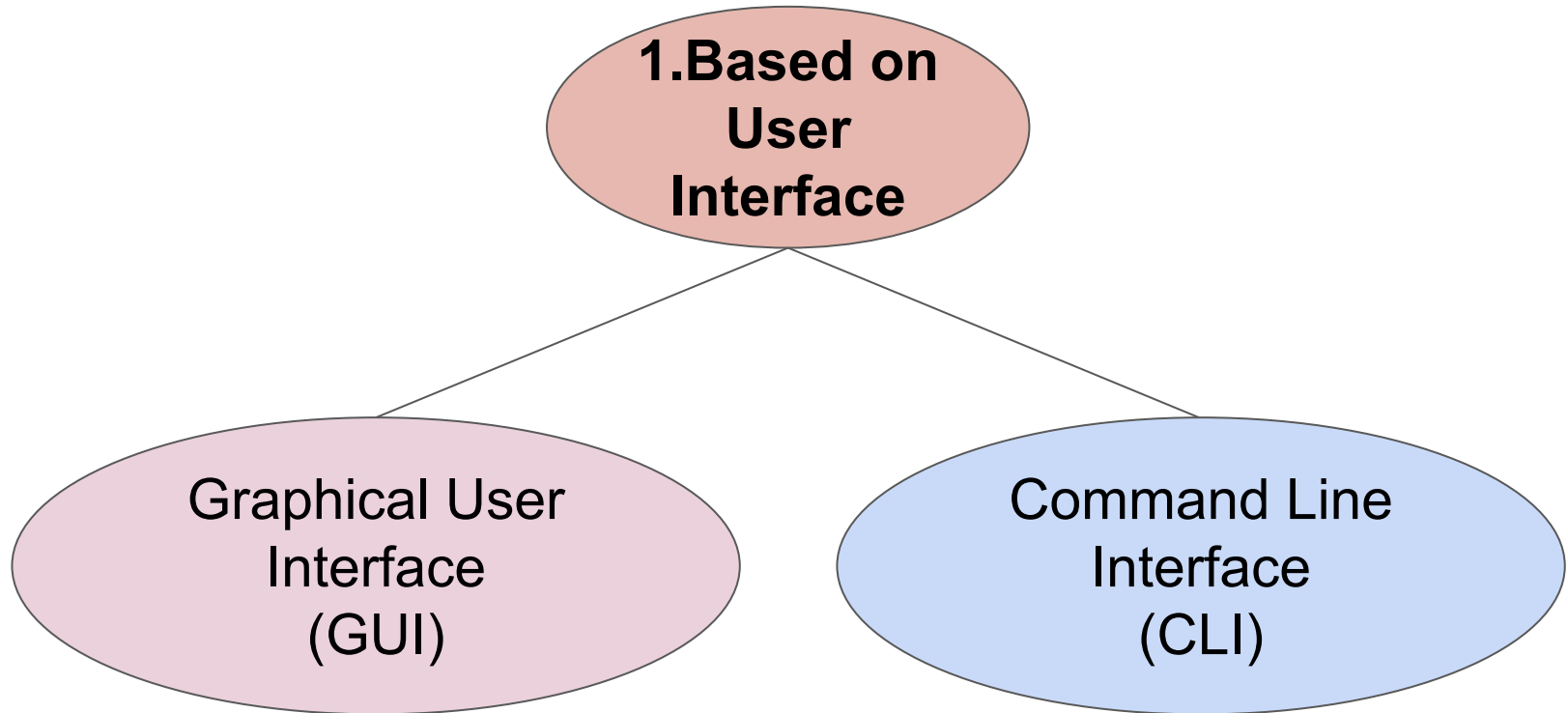
6. Importance of Operating System

- An operating system is essential for **starting and running a computer**.
- It acts as an **interface between the user and computer hardware**.
- It allows users to **run application software** like Word, browsers, and pharmacy software.
- It **manages computer resources** such as CPU, memory, and storage.
- It helps in **file and data management**, including storing and retrieving information.
- It supports **multitasking**, allowing multiple programs to run at the same time.
- It provides **security and data protection** through passwords and access control.
- It controls **input and output devices** like keyboard, printer, scanner, and medical devices.

7. Types of operating systems



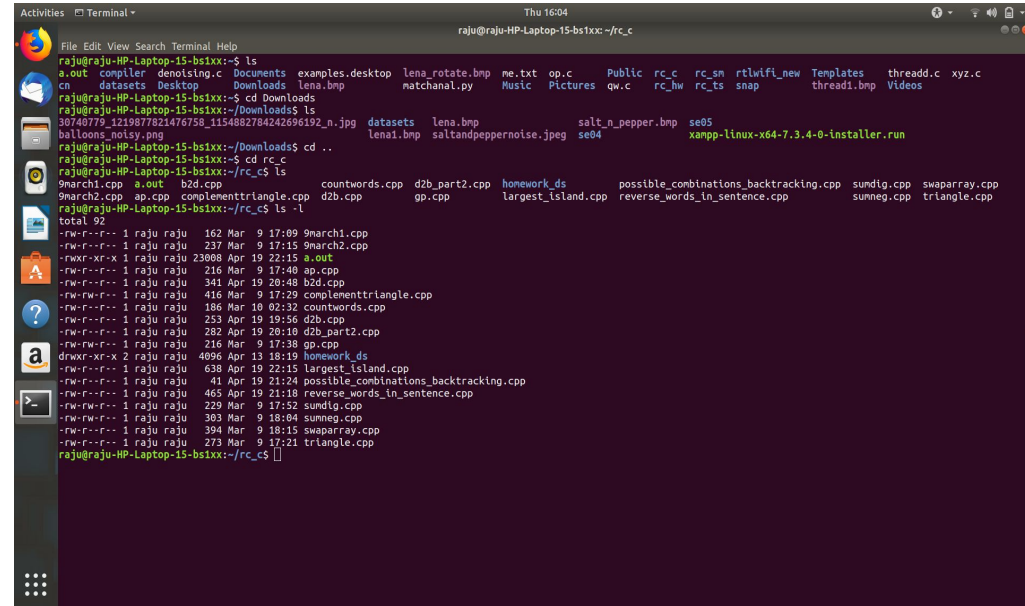
7. Types of operating systems



7. Types of operating systems



GUI



CLI

7. Types of operating systems

Aspect	GUI	CLI
Interface Type	Graphical (icons, windows, menus)	Text-based (commands)
Input Method	Mouse / Touch	Keyboard
Ease of Use	Easy for beginners	Difficult for beginners
Speed (Expert User)	Slower	Faster
Resource Usage	High	Low
Automation	Limited	Strong (scripts)
Control Level	Limited	Very powerful
Typical Users	General users	Programmers / Admins
Examples	Windows, macOS	Bash, CMD

7. Types of operating systems

2.Based on Task

1. Batch Operating System
2. Multiprogramming Operating System
3. Time-Sharing/Multitasking Operating System
4. Multiprocessor Operating System
5. Distributed Operating System
6. Real-Time Operating System (RTOS)
7. Mobile Operating System
8. Desktop/Personal Computer Operating System

7. Types of operating systems

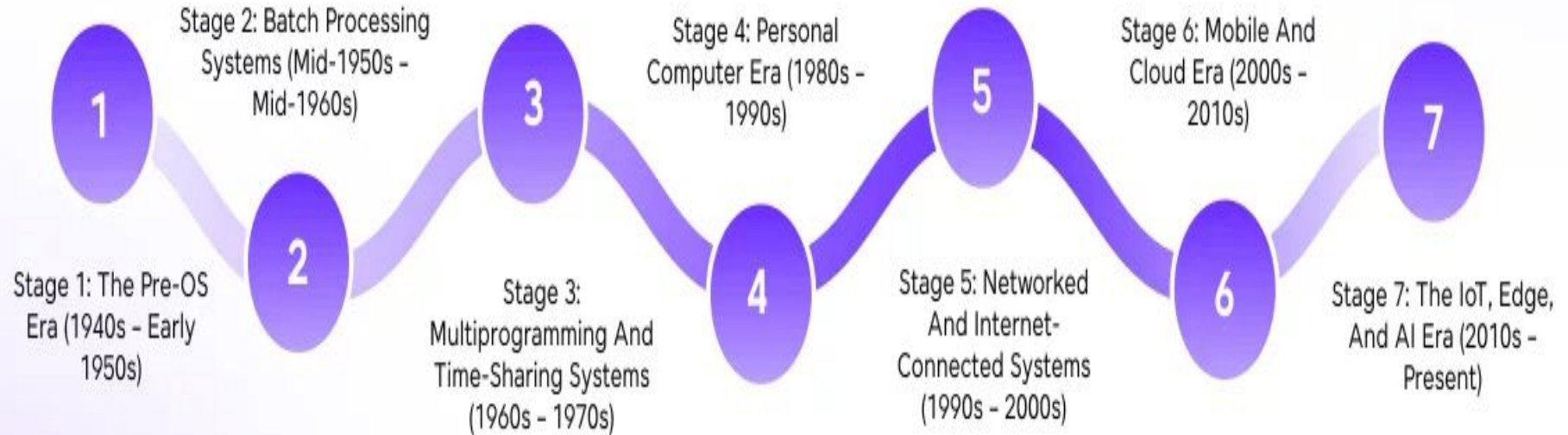
OS Type	Main goal / focus	Key characteristics (short)	Typical examples / deployments	Primary difference vs closest related OS
Batch OS	Maximize throughput for queued jobs	Jobs collected & run in batches; no interactive I/O; FIFO/priority scheduling; long turnaround, low response need	Historical mainframes (IBM OS/360); modern nightly ETL, payroll jobs	Non-interactive and queue-driven vs Multiprogramming which overlaps CPU/I/O for utilization
Multiprogramming OS	Keep CPU busy / improve utilization	Multiple programs resident in memory; switch on I/O waits; requires memory & job scheduling	Early UNIX variants, server kernels (conceptual basis in modern OSes)	Overlaps work on a single CPU vs Multiprocessor which uses multiple CPUs for parallelism
Time-Sharing / Multitasking OS	Provide interactive responsiveness to many users/processes	Short time-slices, preemptive scheduling (RR, multi-level), frequent context switches; fairness & low response time	Multi-user UNIX, interactive desktop OS modes (Windows/macOS/Linux)	Tuned for fast interactive response vs RTOS which enforces strict timing guarantees
Multiprocessor OS	Scale performance via parallel CPUs/cores	Manages multiple CPUs/cores, load balancing, synchronization, SMP/AMP variants; true parallel execution	Multi-core servers/desktops running Linux/Windows Server; HPC nodes	Parallel execution on shared-memory machine vs Distributed OS across networked machines

7. Types of operating systems

OS Type	Main goal / focus	Key characteristics (short)	Typical examples / deployments	Primary difference vs closest related OS
Distributed OS	Present many networked machines as a single coherent system	Resource/location transparency, distributed file/IPC, fault tolerance, complex consistency & comms	Cluster middleware, some research OSs; application-level: HDFS/GFS, Kubernetes (resource manager)	Hides machine boundaries and uses networked resources vs Network OS that exposes separate machines
Real-Time OS (RTOS)	Guarantee deterministic timing (deadlines)	Deterministic/ bounded latencies; priority / EDF / rate-monotonic scheduling; small footprint; hard vs soft RT constraints	FreeRTOS, VxWorks, QNX — automotive controllers, avionics, medical devices	Provides strict timing guarantees vs Desktop/Time-sharing OS that prioritize throughput/responsiveness, not deadlines
Mobile OS	Power-efficient, sensor-rich device support	Aggressive power & memory management, touch/UI frameworks, sandboxing, permissions, cellular stacks	Android, iOS, watchOS, wearOS on phones/tablets/wearables	Optimized for battery, sensors and app-store model vs Desktop OS which expects richer hardware & peripherals
Desktop / Personal Computer OS	General-purpose productivity & multimedia	Rich GUI, broad hardware support, virtual memory, multi-tasking, user accounts	Microsoft Windows, Apple macOS, Linux desktop distros	Optimized for powerful hardware, peripherals and user apps vs Mobile OS which is constrained and touch-first

8. Evolution of Operating System

Evolution means how operating systems **developed step by step over time** to become more advanced and user-friendly.



9. Why Operating System Knowledge Is Essential

Knowledge of Operating Systems (OS) is essential not only for computer professionals but also for chemists, medical practitioners, and non-technical researchers because modern scientific and medical work is computer-driven. Some cases are:

1. Efficient Use of Scientific & Medical Software

- Chemists use software for molecular modeling, spectroscopy, chromatography, and data analysis.
- Medical practitioners use hospital information systems, imaging software (CT, MRI), EHRs, and diagnostic tools.
- ***Understanding OS concepts helps users:***
 - Install, update, and configure software correctly
 - Avoid crashes, compatibility issues, and data loss

Example: A chemist running simulations on Linux needs to manage files, memory usage, and background processes efficiently.

9. Why Operating System Knowledge Is Essential

2. Data Management & File Organization

- Research generates large volumes of data.
- ***OS knowledge helps in:***
 - Organizing datasets systematically
 - Managing file permissions and access
 - Backups and recovery

Example: A medical researcher must securely store patient data while ensuring only authorized access (HIPAA-like requirements).

3. Multitasking & Productivity

- ***Operating systems manage:***
 - Multiple applications running simultaneously
 - Resource allocation (CPU, memory, storage)

Example: A doctor may run patient records, diagnostic software, and video consultation tools at the same time without system slowdown.

9. Why Operating System Knowledge Is Essential

4. Security & Privacy Awareness

- Sensitive data (patient records, experimental results) must be protected.
- ***OS knowledge helps users understand:***
 - User accounts and permissions
 - Malware risks and updates
 - Encryption and secure access

Example: A practitioner avoiding malware by recognizing suspicious processes or unsafe installations.

5. Research Reproducibility & Reliability

- Many experiments depend on OS versions, software environments, and system settings.
- ***Understanding OS helps ensure:***
 - Reproducible experiments
 - Proper documentation of environments

Example: A chemistry simulation may produce different results if run on different OS versions without proper configuration.

6. Use of Advanced Computing Resources

- Non-technical researchers increasingly use: Cloud computing, High-performance computing (HPC), Remote servers

OS knowledge helps in:

- Logging into remote systems
- Running jobs
- Managing storage and permissions

Example: A biomedical researcher submitting analysis jobs to a Linux-based research server.

7. Troubleshooting & Independence

- Basic OS knowledge allows professionals to:
 - Fix minor system issues
 - Understand error messages
 - Communicate effectively with IT staff

Example: A lab scientist identifying whether an issue is software-related or hardware-related before reporting it.

End of Chapter 4